LAB ASSIGNMENT 02
OBJECT ORIENTED PROGRAMMING

Name : Mahak Goyal
Roll No. :2026010066
Course : Object oriented programming with C++
Group : 1MCA3

1. **Write a C++ program to create a class Student with data members name and rollNo. Use a default constructor to initialize these values and display the student details.**

```
#include <iostream>
using namespace std;

class Student {
private:
 string name;
   int rollNo;
public:
   Student()
   {
      name = "Mahak";
      rollNo = 101;
   }
   void display()
   {
      cout << "Student Details" << endl;
      cout << "Name: " << name << endl;
      cout << "Roll No: " << rollNo << endl;
   }
};
int main()
{
   Student s;
   s.display();
   return 0;
}
```

```
Student Details
Name: Mahak
Roll No: 101

--------------------------------
Process exited after 5.785 seconds with return value 0
Press any key to continue . . .
```

2. **Create a class Rectangle having data members length and breadth. Use a parameterized constructor to initialize the values and display the area of the rectangle.**

```
#include <iostream>
using namespace std;

class Rectangle {
```

```
private:
   int length;
   int breadth;
public:
   Rectangle(int l, int b)
   {
      length = l;
      breadth = b;
   }
   void display()
   {
      int area = length*breadth;
      cout << "Length: "<< length<< endl;
      cout << "Breadth: "<< breadth<< endl;
      cout << "Area: "<< area<< endl;
   }
};
int main()
{
   Rectangle r(10, 5);
   r.display();
   return 0;
}
```

```
Length: 10
Breadth: 5
Area: 50

--------------------------------
Process exited after 0.1696 seconds with return value 0
Press any key to continue . . .
```

3. **Write a C++ program to create a class Employee with data members name, id, and salary. Initialize the data members using a parameterized constructor and display the employee details.**

```
#include <iostream>
using namespace std;

class Employee {

private:
 string name;
   int id;
   float salary;
public:
 Employee(string n, int i, float s)
   {
      name = n;
      id = i;
      salary = s;
   }
   void display()
   {
      cout << "Employee Details"<< endl;
      cout << "Name: "<< name<< endl;
```

```
        cout << "ID: "<< id<< endl;
        cout << "Salary: "<< salary<< endl;
    }
};
int main()
{
    Employee e("Mahak", 101, 50000);
    e.display();
    return 0;
}
```

```
Employee Details
Name: Mahak
ID: 101
Salary: 50000

--------------------------------
Process exited after 6.319 seconds with return value 0
Press any key to continue . . .
```

4. **Create a class Box with data members length, width, and height. Implement:**
   · a default constructor,
   · a parameterized constructor,
   · a function to calculate and display volume

```
#include <iostream>
using namespace std;

class Box {
private:
    int length;
    int width;
    int height;
public:
    Box()
    {
        length = 1;
        width = 1;
        height = 1;
    }
    Box(int l, int w, int h)
    {
        length = l;
        width = w;
        height = h;
    }
 void displayVolume()
    {
        int volume = length * width * height;
        cout << "Length: "<< length<< endl;
        cout << "Width: "<< width<< endl;
        cout << "Height: "<< height<< endl;
        cout << "Volume: "<< volume<< endl;
    }
};
```

```
int main()
{
   Box b1;
   Box b2(10, 5, 4);
   cout << "Default Constructor:" << endl;
    b1.displayVolume();
    cout << "\nParameterized Constructor:" << endl;
   b2.displayVolume();
   return 0;
}
```

```
Default Constructor:
Length: 1
Width: 1
Height: 1
Volume: 1

Parameterized Constructor:
Length: 10
Width: 5
Height: 4
Volume: 200

--------------------------------
Process exited after 0.1604 seconds with return value 0
Press any key to continue . . .
```

5. **Write a program to demonstrate constructor overloading using a class Student. Create:**
   - **a default constructor,**
   - **a constructor accepting name and roll number,**
   - **a constructor accepting name, roll number, and CGPA.**

   Display the details of objects created using all three constructors.

```
#include <iostream>
using namespace std;

class Student {
private:
 string name;
   int rollNo;
   float CGPA;

public:
   Student()
   {
      name = "Unknown";
      rollNo = 0;
      CGPA = 0;
   }
   Student(string n, int r)
   {
      name = n;
      rollNo = r;
      CGPA = 0;
   }
```

```
    Student(string n, int r, float c)
    {
        name = n;
        rollNo = r;
        CGPA = c;
    }
    void display()
    {
        cout << "Name: "<< name<< endl;
        cout << "Roll No: "<< rollNo<< endl;
        cout << "CGPA: "<< CGPA<< endl;
    }
};
int main()
{
    Student s1;
    Student s2("Mahak", 101);
    Student s3("Mahak", 101, 8.5);
    cout << "Default Constructor:"<< endl;
    s1.display();
 cout << "\nConstructor with Name & Roll No:"<< endl;
    s2.display();
 cout << "\nConstructor with Name, Roll No & CGPA:"<< endl;
    s3.display();
    return 0;
}
```

```
Default Constructor:
Name: Unknown
Roll No: 0
CGPA: 0

Constructor with Name & Roll No:
Name: Mahak
Roll No: 101
CGPA: 0

Constructor with Name, Roll No & CGPA:
Name: Mahak
Roll No: 101
CGPA: 8.5
```

6. **Create a class Complex with real and imaginary data members. Implement:**
   ·a default constructor,
   ·a parameterized constructor,
   ·a display function.

```
#include <iostream>
using namespace std;

class Complex {
private:
    int real;
 int imaginary;
public:
Complex()
```

```
    {
        real = 0;
        imaginary = 0;
    }
 Complex(int r, int i)
    {
        real=r;
        imaginary=i;
    }

    void display()
    {
        cout<<real<< " + "<< imaginary<< "i"<< endl;
    }
};
int main()
{
    Complex c1;
 Complex c2(5, 3);
    cout<<"Default Constructor: ";
    c1.display();
 cout<<"Parameterized Constructor: ";
    c2.display();
    return 0;
}
```

```
Default Constructor: 0 + 0i
Parameterized Constructor: 5 + 3i

--------------------------------
Process exited after 0.1495 seconds with return value 0
Press any key to continue . . .
```

7. **Create a class Demo that displays a message when its constructor is called and another message when its destructor is called. Create multiple objects of the class and observe the order in which constructors and destructors execute.**

```
#include <iostream>
using namespace std;

class Demo {
public:
Demo() {
        cout<< "constructor called"<< endl;
    }
 ~Demo() {
        cout<< "destructor called"<< endl;
    }
};
int main() {
Demo d1;
Demo d2;
Demo d3;
    return 0;
}
```

```
constructor called
constructor called
constructor called
destructor called
destructor called
destructor called

--------------------------------
Process exited after 0.3523 seconds with return value 0
Press any key to continue . . .
```

8. **Write a C++ program to demonstrate the execution order of constructors and destructors when objects are created inside main() and inside a separate function.**

```
#include <iostream>
using namespace std;

class Demo {
public:
Demo() {
        cout<< "Constructor called"<< endl;
    }
 ~Demo() {
        cout<< "Destructor called"<< endl;
    }
};
void test() {
    cout<< "\nInside separate function:"<< endl;
 Demo d2;
    cout<< "Function is ending"<< endl;
}
int main() {
 cout<< "Inside main:"<< endl;
Demo d1;
    test();
 cout<< "\nMain is ending"<< endl;
    return 0;
}
```

```
Inside main:
Constructor called

Inside separate function:
Constructor called
Function is ending
Destructor called

Main is ending
Destructor called

--------------------------------
Process exited after 0.3565 seconds with return value 0
Press any key to continue . . .
```